

Problèmes avec concurrence = parallélisme

- Les threads concurrents doivent parfois partager des ressources et des données (fichiers ou mémoire commune)
 - On parle donc de threads *coopérants*
- Si l'accès n'est pas contrôlé, le résultat de l'exécution du programme pourra dépendre de l'ordre d'entrelacement de l'exécution des instructions (*non-déterminisme*).
- Un programme pourra donner des résultats différents et parfois indésirables de fois en fois.

Exemple1

Deux threads exécutent cette même procédure et partagent la même base de données.

Ils peuvent être interrompus n'importe où.

Le résultat de l'exécution concurrente de P1 et P2 dépend de l'ordre de leur entrelacement.

**M. X demande une
réservation
d'avion**

**Base de données
dit que fauteuil
A est disponible**

**Fauteuil A est
assigné à X et
marqué occupé**

Exemple1

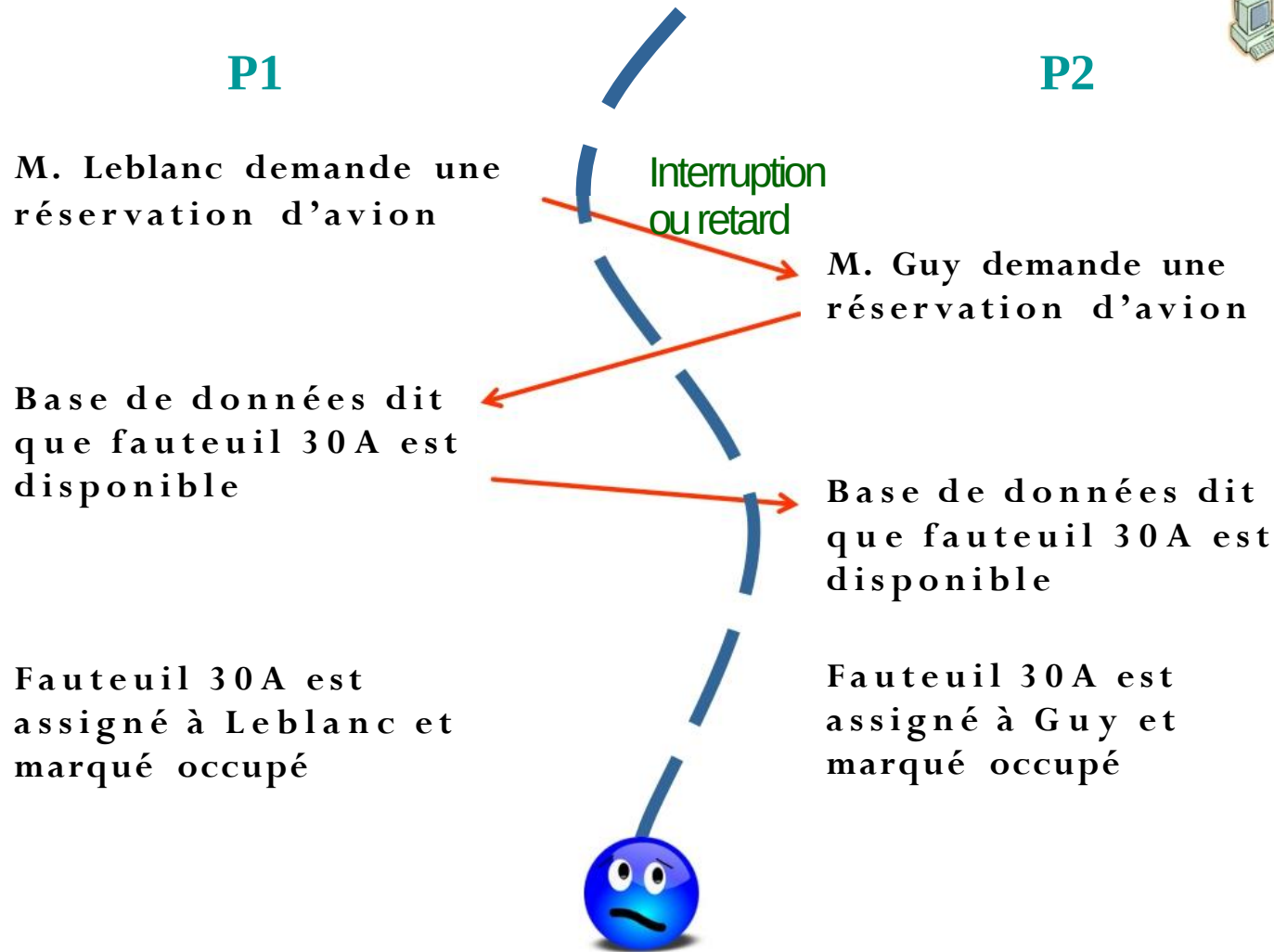


Figure : Vue globale d'une exécution possible

Exemple2

Processus P1

```
int a;  
  
void main()  
{  
....  
    scanf("%d",&a)  
    printf("%d",a)  
}
```

Processus P2

```
int a;  
  
void main()  
{  
....  
    scanf("%d",&a)  
    printf("%d",a)  
}
```

- Si la var a est partagée, le premier a est effacé.
- Si elle est privée, l'ordre d'affichage est renversé

Asynchronie des threads

- Quand plusieurs threads exécutent en parallèle, nous ne pouvons pas faire d'hypothèses sur la vitesse d'exécution des threads, ni leur entrelacement.
- Le partage d'objets sans précaution particulière peut conduire à des résultats imprévisibles (incohérents).
- La solution à ce problème s'appelle **synchronisation des processus**.

Synchronisation interprocessus

- Problème de l'exclusion mutuelle
- Synchronisation
 - o Solutions matérielles
 - o Solutions logicielles

Définitions

- **La synchronisation** : C'est le contrôle de l'ordre d'exécution des tâches
- **Ressources critiques** :
 - Les ressources partagées, comme des zones mémoire, périphériques d'entrées/sorties, ..., sont dites critiques si elles ne peuvent être utilisées simultanément que par un seul processus.
- **Exemple** : Deux processus P et Q produisent des données devant être imprimées sur une imprimante unique. L'emploi de cette imprimante par P exclut l'emploi de l'imprimante par Q tant que l'impression par P n'est pas terminée
 - L'imprimante est une ressource critique : Un seul point d'accès

Définitions

- Section critique
 - Une SC est une suite d'instructions qui opèrent sur une ou plusieurs ressources partagées et qui nécessitent une utilisation exclusive de ces ressources (ressources critiques).
 - Une section critique (SC) est un ensemble d'instructions d'un programme qui peuvent engendrer des résultats imprévisibles (ou incohérents) lorsqu'elles sont exécutées simultanément par des processus différents.

Interblocage (Deadlock)

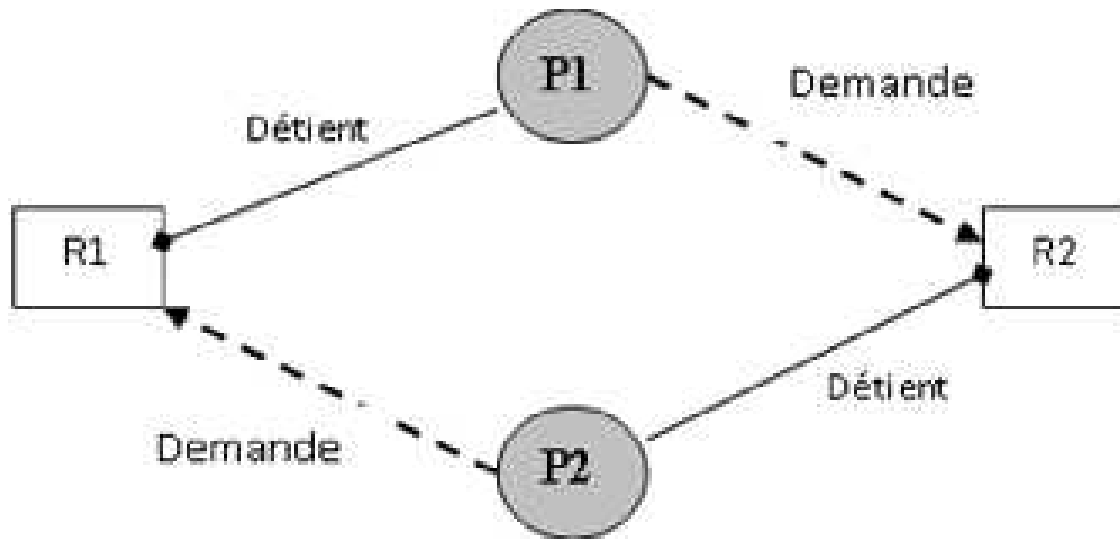
- Un ensemble de processus est dans une situation d'interblocage si chaque processus de l'ensemble attend un événement qui ne peut être produit que par un autre processus de l'ensemble.
- Autrement, l'interblocage est vu comme étant un état anormal où, au moins, deux processus se trouvent incapables de poursuivre leurs exécutions. Cet état est caractérisé par le fait que chaque processus est bloqué et ne peut être débloqué que par un autre processus de l'ensemble.

Interblocage (Deadlock)

- Une situation d'interblocage peut survenir si les quatre conditions suivantes se produisent simultanément :
 - 1- Accès exclusif : Les ressources ne peuvent être exploitées que par un seul processus à la fois.
 - 2- Attente et occupation : Les processus qui demandent de nouvelles ressources gardent celles qu'ils ont déjà acquises et attendent la satisfaction de leur demande
 - 3- Pas de réquisition : Les ressources déjà allouées ne peuvent pas être préemptées.
 - 4- Attente circulaire : Les processus en attente des ressources déjà allouées forment une chaîne circulaire d'attente.

Exemple d'interblocage

Un système possède une instance unique de chacun des deux types de ressources R1 et R2. Un processus P1 détient l'instance de la ressource R1 et un autre processus P2 détient l'instance de la ressource R2. Pour suivre son exécution, P1 a besoin de l'instance de la ressource R2, et inversement P2 a besoin de l'instance de la ressource R1. Une telle situation est une situation d'interblocage.



Exclusion mutuelle

- Pour assurer l'exécution des sections critiques des processus, il est nécessaire de trouver une solution qui permet d'éviter les conditions de concurrence.
- L'exclusion mutuelle est une méthode qui permet de s'assurer que si un processus utilise une variable ou une ressource partagée, les autres processus seront exclus de la même activité.

Exclusion mutuelle

- Une instruction de type exclusion mutuelle se décompose en 3 étapes

Section non critique.

Protocole d'entrée en section critique.

Section critique.

Protocole de sortie de la section critique.

Section non critique.

- Les deux protocoles d'entrée et de sortie permettent de contrôler l'obtention et la libération du droit d'accès en exclusion mutuelle par un processus donné
- Protocole d'entrée en SC: ensemble d'instructions qui permet cette vérification et la non progression éventuelle;
- Protocole de sortie de SC: ensemble d'instructions qui permet à un processus ayant terminé sa SC d'avertir d'autres processus en attente que la voie est libre

Principe de l'exclusion mutuelle

Certaines conditions doivent être vérifiées pour toute solution de synchronisation :

1. **Exclusion mutuelle** : A tout instant, un processus au plus peut se trouver en section critique
2. **Pas de famine**
3. **Pas d'interblocage** : Si un processus est bloqué hors sa section critique, ce blocage ne doit pas empêcher l'entrée d'un autre processus en sa section critique
4. **Uniformité** : La solution doit être la même pour tous les processus c-à-d, qu'aucun processus ne doit jouer le rôle privilégié

=> **Comment assurer alors une coopération correcte et efficace des processus ?**

Solution pour assurer l'exclusion mutuelle

- Solutions de niveau Matériel :
 - Masquage des interruptions
 - Instruction indivisible, TAS (Test and Set)
- Solutions logicielles :
 - Alternance, Peterson
 - Dijkstra (Semaphore)

Masquage des interruptions

- Protocole d'entrée en SC : masquer les interruptions au niveau du CPU (interruption d'horloge qui permet d'interrompre un processus lorsqu'il a épuisé son quantum).
- La SC ne peut pas être interrompue car les changements de contexte sont impossibles.
- Protocole de sortie de SC : démasquer les interruptions.

Masquer les IT
SC
Démasquer les IT ;

Masquage des interruptions

Remarques :

- Le masquage des IT n'est accessible qu'aux programmeurs privilégiés pour des raisons de fiabilité comme le super-utilisateur.
- Ne convient pas pour les machines multi-processeurs à mémoire partagée (car chaque processeur gère ses propres interruptions).

Analyse :

- Absence de blocage : l'ordonnanceur du CPU finit par donner la main à un processus voulant rentrer dans sa SC.
- Condition de progression : une tâche doit pouvoir entrer en SC si aucune autre ne s'y trouve.
- Absence de famine : si l'ordonnanceur l'assure (normalement oui !).
- **Solution dangereuse en mode utilisateur :**
 - Si dans un processus, le programmeur a oublié de restaurer les interruptions, c'est la fin du système ☹

Instruction indivisible, TAS (Test and Set)

- Les ordinateurs multiprocesseurs ont une instruction atomique indivisible appelée **TAS** (Test and Set Lock).
- Lorsqu'un processeur exécute l'instruction **TAS**, il **verrouille le bus de données** pour empêcher les autres processus d'accéder à la mémoire pendant la durée de l'opération.

Si un processus est entrain d'effectuer un test-and-set sur un emplacement mémoire

Si plusieurs processus tentent d'accéder à la même mémoire



Aucun autre processus ne peut commencer un autre test-and-set jusqu'à ce que le premier processus soit terminé

TAS : Protéger l'espace mémoire en cas d'accès concurrents

Instruction indivisible, TAS (Test and Set)

- Cette instruction peut être utilisée pour assurer l'exclusion mutuelle.
- Voilà d'une façon simplifier l'esprit de cette méthode :

verrou: variable commune, initialement 0 (« déverrouillé »)

Si verrou = 0 alors
TAS met verrou à 1
et retourne 0 dans
la variable prev
Sinon TAS retourne
la val de verrou (1)

P1

```
prev : var locale
...
prev = TAS(verrou);
Tant que (prev = 1)
Faire
    /* sleep(délai); */
    prev = TAS(verrou);
Fintantque
    <section critique>
    verrou = 0;
...
```

P2

```
prev : var locale
...
prev = TAS(verrou);
Tant que (prev = 1)
Faire
    /* sleep(délai); */
    prev = TAS(verrou);
Fintantque
    <section critique>
    verrou = 0;
...
```

- Cette solution est acceptable dans un système multiprocesseur et si l'exclusion mutuelle survient rarement.

Alternance

- Utiliser une variable tour qui mémorise le tour du processus qui doit entrer en section critique. Tour est initialisée à 0.

- Protocole d'entrée en SC :
while (tour != monNumero) ; // attente active
- Protocole de sortie de la SC :
tour=(monNumero +1) % N ; // au suivant !

N est le nombre de processus qui souhaite exécuter la SC

Alternance

Rien faire

Thread T0:

`while(tour!=0);`

`// premier à entrer`

`SC`

`tour = 1;`

`nonSC`

`while(tour!=0);`

`// entre quand T1 finit`

`SC`

`tour = 1;`

`nonSC`

`etc...`

Thread T1:

`while(tour!=1);`

`// entre quand T0 finit`

`SC`

`tour = 0;`

`nonSC`

`while(tour!=1);`

`// entre quand T0 finit`

`SC`

`tour = 0;`

`nonSC`

`etc...`